LCI workshop
on high performance clustered computing

LCI Advanced Workshop 2025: **Job Submit Plugin**

Willy Dizon
Senior Systems Analyst
Arizona State University
willy.dizon@asu.edu

ASU Research Computing

Job Submit Plugin Overview

- Introduction and Objectives
  - What is the goal?
  - Who will use these?
  - Potential uses
  - Should I deploy JSP?
- Fundamentals Recap
  - Plugin Mental Model
  - Enabling the Job Submit Plugin
- Understanding JSP
  - by Examples
  - Developing and Debugging
  - More Gotchas
  - Guarding Against High Values
  - Available Fields
  - Fitting your Code around Edge cases
  - Other Ideas

What is the Goal?

The Job Submit Plugin enables a programmatic approach to modifying jobs at submission time.

It allows concrete business logic (site policy) to meticulously modify resource requests in a profoundly more comprehensive way than the `slurm.conf` config permits.

Who Uses the Job Submit Plugin?

The Job Submit Plugin (JSP) should have broad appeal to any site whose cluster's usage policy is more complex than "group X owns resource Y" or "all users can use anything".

Programmatically implementing new kinds of resource limits, specific job rejection, or creating precise resource defaults are all examples of reasons you might explore the JSP's capabilities.

Potential Uses

- Ensure some resources are always requested in tandem, e.g., a GPU with a given amount of memory.
- Reject specific jobs matching a specific request of resources
- Create secondary “partitions”--or groups of nodes more sophisticated or dynamic than built-in partitions
- Auto-fill some slurm directives for users, e.g., mail-to address
- Revise requests for allocations to more appropriate resources/partitions than requested

Should I Deploy Job Submit Plugin?

PROS:

- Precise programmatic control of submissions
- Allows implementation of measures built-in Slurm cannot
- Can help reduce number of submitted-but-unrunnable jobs
- Can produce very helpful slurmctld logs
- *Can provide instructions or information to user at submission time*

CONS:

- LUA is finicky; poorly-written code that errors out will skip the remaining JSP execution entirely!
- LUA or C-written JSP might be completely unserviceable by other members on the team
- JSP modifications might defy the expectations of users (if not well-documented)

Fundamentals Recap

PLUGIN MENTAL MODEL

1. Job is submitted by user via `sbatch`, `salloc`, etc.
2. Execution occurs on the slurmctld host with the `job_submit.lua` file
3. Job characteristics are parsed and put into a LUA object
4. Execution progresses from `function slurm_job_submit()` of JSP file
5. Code modifies the LUA object (job characteristics) per your requirements
6. Valid execution through the *code* returns a zero or nonzero exit code
7. On `0` (zero), job successfully accepted, non-zero the job submission rejected

Enabling the Job Submit Plugin

The Job Submit Plugin can be written in C or in LUA. Depending on the expertise of your team, the complexity of your site policy, and language preference, either can be used with near-exact functionality parity.

This slide deck will focus on LUA-based JSP only.

To enable, make this change in `slurm.conf`:

```
JobSubmitPlugins = lua
```

Changes to `slurm.conf` require an `scontrol reconfig`, but changes to `job_submit.lua` happen *immediately.*

Understanding the Job Submit Plugin by Examples

EXAMPLE #1: auto-populating user email address

In our wiki, we have easy copy-pastable SBATCH templates including directives such as:

```
#SBATCH --mail-user=%u@asu.edu      # edit this for your own username!
```

Many users lazily copy-paste without examining and making the required change. We can choose to have this behavior automatically rectified:

```
--PURPOSE: Stage2.7 detect template value %u for mail user and substitute

if job_desc.mail_user == "%u@asu.edu" then
    --template value should be changed in all circumstances, mailtype kept intact
    job_desc.mail_user = job_desc.user_name .. "@asu.edu"
end
```

Understanding the Job Submit Plugin by Examples (cont.)

EXAMPLE #2: minimum system memory with request of GPU

GPUs can benefit from RAM matching/near-matching VRAM amount. This is often overlooked by users whose request ends up with 80GB VRAM, RAM at default 2GB.

```
#SBATCH --gres=gpu:a100:1
#SBATCH --mem=2G     # default, if not specified
```

This will result in a bottleneck that will generally confuse a user who *knows* their model fits within 80GB but is nonetheless running into memory issues.

```
-- PURPOSE: Stage2.8 - Ensure minimum system memory for GPU jobs if omitted
if (not job_desc.min_mem_per_node or job_desc.min_mem_per_node == 0)
   and (has_gres or has_tres) then
     -- apply default GPU memory requirement
     job_desc.min_mem_per_node = 24000
     slurm.log_user("Default system mem w/ GPU: " .. job_desc.min_mem_per_node .. " MB")
     slurm.log_info("[Stage2.8] min_mem_per_node set to %u", job_desc.min_mem_per_node)
end
```

This is indicated to the user on their submission terminal as well as the slurmctld `/var/log/slurmctld.log` file.

Understanding the Job Submit Plugin by Examples (cont.)

EXAMPLE #3: auto-populate defaults for omitted characteristics

```
-- PURPOSE: Stage2.0 - Apply defaults for QOS, time_limit
if not job_desc.qos then
    job_desc.qos = defaults.qos
    slurm.log_user("Default QOS applied: " .. job_desc.qos)
    slurm.log_info("[Stage2] qos set to '%s'", job_desc.qos)
end
local tl = tonumber(job_desc.time_limit)
if not tl or tl <= 0 or tl > 4e9 then
    job_desc.time_limit = defaults.time_limit
    slurm.log_user("Default time_limit applied: " .. job_desc.time_limit .. " minutes")
    slurm.log_info("[Stage2] time_limit set to %u", job_desc.time_limit)
    tl = job_desc.time_limit
end
```

Take note, in these examples, the value is no longer hard coded but instead reference `defaults.***` . These values can be refactored out of the code itself to permit immediate changes and easier documentation. (cont.)

Understanding the Job Submit Plugin by Examples (cont.)

EXAMPLE #3 CONTINUED: auto-populate defaults for omitted characteristics

```
# head job_submit.lua
-- PURPOSE: Load configuration and defaults
local slurm_cfg                  = dofile("/etc/slurm/slurm_cfg.lua")
local defaults                   = slurm_cfg.defaults or {}

# head -n 20 slurm_cfg.lua
return {
    -- Default values for omitted job characteristics
    defaults = {
        qos           = "public",  -- (see 2.0)
        partition     = "public",  -- (see 1.0)
        time_limit    = 240,       -- 4 hours  (see 2.0)
        cpus_per_task = 1,         -- (see 2.0)
        min_mem_gpu   = 24000,     -- in MB  (see 2.8)
        partition_map = {          -- (see 2.0)
            short  = "htc",          -- ≤240 min
            medium = "public"        -- >240 min
        }
    }
```

Understanding the Job Submit Plugin by Examples (cont.)

EXAMPLE #4: informing users of resource allocation changes

```
-- PURPOSE: Stage 1.5; alert user for any request of a license
if job_desc.licenses ~= nil then
    slurm.log_user('Slurm licenses has been deprecated and no longer affect resource
allocation. You can remove all requests for licenses from your scripts/requests. See
https://mywiki.example.com/requesting_grace_hopper_nodes')
    --job_desc.licenses = nil
    --uncomment to to ignore it for them & accept job, or just advise removing
end
```

The Job Submit Plugin can in many cases save one additional ping on your Slack or Teams channel requesting help for why a script that worked months ago now is getting some sort of pushback.

Developing and Debugging

#1 START SMALL!

This is a bare bones job_submit.lua that lets jobs pass as submitted.

```
# cat job_submit.lua
-- job_submit.lua
function slurm_job_submit(job_desc, part_list, submit_uid)
    return slurm.SUCCESS
end

function slurm_job_modify(job_desc, job_rec, part_list, modify_uid)
    return slurm.SUCCESS
end
```

Developing and Debugging

#2 CODE DEFENSIVELY

Call out **exceptionally small cases to modify**.

```
# head job_submit.lua
-- job_submit.lua
function slurm_job_submit(job_desc, part_list, submit_uid)
    -- PURPOSE: Any license request necessitates --exclusive allocation
    if job_desc.licenses then
        job_desc.shared = 2  -- node becomes available to jobs by user only
    end
end
…
```

Design modifications so all jobs run through all checks, top to bottom, for clarity of tracing.

Developing and Debugging

#3 INVESTIGATE JOB_DESC DATATYPE

```
slurm.log_info("%s", job_desc.partition)
```

in job_submit.lua… then requesting the allocation:

```
$ salloc -p general
```

…reports in `/var/log/slurmctld.log`

```
[2025-09-24T12:56:09.195] lua: general
```

so this would work:

```
if job_desc.partition == “general”
    slurm.log_user("you’re submitting to partition: %s", job_desc.partition)
end if
```

Developing and Debugging

#4 UNDERSTAND JOB_DESC DATATYPE

```
slurm.log_info("%s", job_desc.partition)
```

in job_submit.lua… then requesting the allocation:

```
$ salloc -p general,htc
```

…reports in `/var/log/slurmctld.log`

```
[2025-09-24T12:56:09.195] lua: general,htc
```

but this would **not** work:

```
if job_desc.partition == “general”
    slurm.log_user("you’re submitting to partition: %s", job_desc.partition)
end if
```

So be mindful that even the most straightforward logic can be inadvertently side-stepped by less-common, but wholly legal and documented operations.

Developing and Debugging

Better String Matching

(this is LUA, really, not Slurm-specific!)

```
$ salloc -p gpu

if job_desc.partition:match("gpu") then
    slurm.log_info("%s", job_desc.partition) -- this will match
end

$ salloc -p intelgpu,amdgpu

if job_desc.partition:match("gpu") then
    slurm.log_info("%s", job_desc.partition) - this will match (!)
end
```

So for clarity, we can change to different datatypes altogether:

Developing and Debugging

Better String Matching (cont.)

Create some helper functions that ease the task of checking and ensure a printable output.

```
-- Return true/false if 'name' is in a comma-separated list 'csv'
local function is_in(name, csv)
    if type(csv) ~= "string" or csv == "" then
        return false
    end
    -- pad with commas to avoid substring matches
    local haystack = "," .. csv .. ","
    local needle   = "," .. name .. ","
    return string.find(haystack, needle, 1, true) ~= nil
end

function slurm_job_submit(job_desc, part_list, submit_uid)
    slurm.log_info("gpu in set? %s", is_in("gpu", job_desc.partition))
    return slurm.ERROR
end
```

Developing and Debugging

Better String Matching (cont.)

```
$ salloc -p gpu

[2025-09-24T12:00:00.000] lua: gpu in set? true

$ salloc -p intelgpu,amdgpu

[2025-09-24T12:00:00.000] lua: gpu in set? false
```

Success! A clean, readable way to match! `is_in("gpu", job_desc.partition)`

This can be readily applied to multiple fields that use CSV-formatting, e.g.,

```
slurm.log_info("%s", job_desc.features or "None")    -- salloc -C gracehopper,arm
slurm.log_info("%s", is_in("gracehopper", job_desc.features))    -- returns true!
```

Developing and Debugging

Syntaxical Errors in JSP

Even the smallest typo can be devastating to the execution of your script. In a built-in effort to combat this, syntaxical errors are caught ahead of time and will default to the last-known working script:

```
[2025-09-24T12:00:00.000] error: job_submit/lua: /etc/slurm/job_submit.lua:
/etc/slurm/job_submit.lua:20: ')' expected (to close '(' at line 19) near 'return', using
previous script
```

```
slurm.log_info("%s" is_in("grace", job_desc.features))
                  ^ whoops, missing comma!
```

This can really cost time, so be mindful to look out if it is ignoring your newly-written logic.

Developing and Debugging

More Gotchas!

job_desc has a lot of attributes that may not behave intuitively. Here's a few attributes and how they might be handled more precisely.

```
$ salloc
```

What value is likely to be filled for `job_desc.cpus_per_task`?

```
slurm.log_info("cpus_per_task? %s", job_desc.cpus_per_task)
```

Developing and Debugging

More Gotchas!

Correct! 65534.0!

```
[2025-09-25T11:14:25.265] lua: cpus_per_task? 65534.0
```

```
$ salloc
```

How about min_nodes and max_nodes?

```
slurm.log_info("min_nodes? %s", job_desc.min_nodes)
slurm.log_info("max_nodes? %s", job_desc.max_nodes)
```

Developing and Debugging

More Gotchas!

Correct! 4294967294.0 (~4.3 billion)

```
[2025-09-25T12:00:00.000] lua: min_nodes? 4294967294.0
[2025-09-25T12:00:00.000] lua: max_nodes? 4294967294.0
```

And the same for `job_desc.time_limit`:

```
slurm.log_info("time_limit? %s", job_desc.time_limit)
```

```
[2025-09-25T12:00:00.000] lua: time_limit? 4294967294.0
```

So how can we guard against these values?

Developing and Debugging

Guarding against High Values

So let's set the following stanza in `slurm.conf`:

```
PartitionName=general \
 DefaultTime=0-04:00:00
```

What should we expect `job_desc.time_limit` to register as now?

```
$ salloc -p general
```

Developing and Debugging

Guarding against ~~High Values~~ Unset Values

```
[2025-09-25T12:00:00.000] lua: time_limit? 4294967294.0
```

Still 4.2 billion for `$ salloc -p general`

We might *think* that time_limit will reflect 4 hours, but job_desc reports back what has been *submitted by the user*, not the default values.

```
if job_desc.time_limit <= 240 then
    slurm.log_info("time_limit is less than or equal to 4 hours")
elseif job_desc.time_limit > 4e9 then
    slurm.log_info("time_limit is unset")
end
```

It's critical to have your logic anticipate unspecified values and to handle them accordingly.

Developing and Debugging

Available Attributes

The list is long, so schedmd points users directly to the source:
https://github.com/SchedMD/slurm/blob/master/src/lua/slurm_lua.c#L613

```
692         } else if (!xstrcmp(name, "gres_req")) {
693             lua_pushstring(L, job_ptr->tres_fmt_req_str);
694         } else if (!xstrcmp(name, "gres_used")) {
695             lua_pushstring(L, job_ptr->gres_used);
696         } else if (!xstrcmp(name, "group_id")) {
697             lua_pushnumber(L, job_ptr->group_id);
698         } else if (!xstrcmp(name, "job_id")) {
699             lua_pushnumber(L, job_ptr->job_id);
700         } else if (!xstrcmp(name, "job_state")) {
701             lua_pushnumber(L, job_ptr->job_state);
702         } else if (!xstrcmp(name, "licenses")) {
703             lua_pushstring(L, job_ptr->licenses);
704         } else if (!xstrcmp(name, "max_cpus")) {
```

Developing and Debugging

Seeing Attributes Live

```
local function log_job_desc(jd)
  local JOB_DESC_FIELDS = {
    "account","acctg_freq","argv","begin_time","burst_buffer",
    "command","comment","constraints","contiguous","core_spec",
    "cpus_per_task","dependency","env_vars","features","gres",
    "group_id","immediate","job_id","licenses","mail_type",
    "mail_user","mem_per_cpu","min_nodes","max_nodes","name",
    "nodes","ntasks","partition","priority","qos","requeue",
    "reservation","script","shared","time_limit","user_id","work_dir"
  }

  for _, k in ipairs(JOB_DESC_FIELDS) do
    local v = jd[k]
    if v ~= nil then
      slurm.log_info("%s = %s", k, tostring(v))
    else
      slurm.log_info("%s = <nil>", k)
    end
  end
end

function slurm_job_submit(job_desc, part_list, submit_uid)
  log_job_desc(job_desc)
  return slurm.SUCCESS
end
```

```
lua: account = <nil>
lua: acctg_freq = <nil>
lua: argv = table: 0x15548003aa
lua: begin_time = 0.0
lua: burst_buffer = <nil>
lua: command = <nil>
lua: comment = <nil>
lua: constraints = <nil>
lua: contiguous = 0.0
lua: core_spec = <nil>
lua: cpus_per_task = 65534.0
lua: dependency = <nil>
lua: env_vars = <nil>
lua: features = <nil>
lua: gres = <nil>
lua: group_id = 0.0
lua: immediate = 0.0
lua: job_id = <nil>
lua: licenses = <nil>
lua: mail_type = 0.0
lua: mail_user = <nil>
lua: mem_per_cpu = <nil>
lua: min_nodes = 4294967294.0
lua: max_nodes = 4294967294.0
```

Developing and Debugging

Fitting your Code around Edge Cases

```
if job_desc.time_limit > 4e9 then
    -- user did not specify time limit
    slurm.log_info("time_limit is unset")
    job_desc.time_limit = 240
    slurm.log_user("Job duration not specified; setting to 4 hours.")
else
    -- user specified time
    if job_desc.time_limit <= 240 then
          if not is_in("htc", job_desc.partition) then
          slurm.log_user("You are requesting a job duration shorter than four hours. You
can increase eligible nodes to your job by using the `htc` partition.")
          end
    end
end
```

It can read much easier separating logic that works on unspecified values. You might also consider handling these sequentially:

Developing and Debugging

Fitting your Code around Edge Cases (cont.)

```
if job_desc.time_limit > 4e9 then
    -- user did not specify time limit
    slurm.log_info("time_limit is unset")
    job_desc.time_limit = 240
    slurm.log_user("Job duration not specified; setting to 4 hours.")
end
… <more detect-and-set-default if/elseif stanzas> …

if job_desc.time_limit <= 240 then
     if not is_in("htc", job_desc.partition) then
     slurm.log_user("You are requesting a job duration shorter than four hours. You can
increase eligible nodes to your job by using the `htc` partition.")
     end
end
```

This example normalizes the values from the start, ensuring “upper limit” values can be safely ignored in the main business logic.

Developing and Debugging

Run Regression Tests

Have a script that submits jobs with various *characteristics you want changed* through the job submit plugin. The accumulation of these tests guarantee the behavior of your plugin.

This code can be written in python, bash, or any other language (not lua!), so long as you are able to probe the Slurm API for the accepted characteristics to check/fail tests.

Regression tests will let you define the behavior of your submission plugin, by legibly displaying the expected outcome of various different submission.

1) Determine a behavior you wish to implement
2) Create a test that accurately depicts the intended behavior
3) Ensure the test fails at the outset!
   If the test doesn't fail, you don't need to write code! OR..
   your test isn't covering all possible edge cases.
4) Write code to handle your new case
5) Ensure the test succeeds!

Developing and Debugging

Run Regression Tests (cont.)

```
238 # UNIT TEST DEFINITION START!
239
240 class TestSlurm(unittest.TestCase):
241     def setUp(self):
242         pass
243
244     def tearDown(self):
245         pass
246
247     @common_slurm_checks
248     @expect_tres({"cpu": 1, "mem": 2, "node": 1})
249     @expect_equal({"partition": "htc"})
250     @expect_equal({"qos": "public"})
251     @expect_equal({"time_limit": 240})
252     def test_sbatch_blank(self):
253         self.details = run('')
254
```

The code depicted here is NOT enough to run tests, but is just an example of how one can clearly and cleanly specify site policies as code.

Other parts of the code not shown here include running subprocesses that run “salloc” and also do Slurm API queries to test the attributes’ actual submission.

Developing and Debugging

Run Regression Tests (cont.)

The test fails! Success!

```
[root@dev-slurm01:slurm]# python3 test_slurm.py
E
======================================================================
ERROR: test_sbatch_blank (__main__.TestSlurm)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_slurm.py", line 168, in wrapper
    func(self, *args, **kwargs)  # Run the test and get details
  File "test_slurm.py", line 215, in wrapper
    func(self, *args, **kwargs)  # Run the test
  File "test_slurm.py", line 184, in wrapper
    func(self, *args, **kwargs)  # Run the test
  File "test_slurm.py", line 184, in wrapper
    func(self, *args, **kwargs)  # Run the test
  File "test_slurm.py", line 184, in wrapper
    func(self, *args, **kwargs)  # Run the test
  File "test_slurm.py", line 253, in test_sbatch_blank
    self.details = run('')
  File "test_slurm.py", line 116, in run
    raise SlurmSubmissionError(parsed[-1], parsed)
SlurmSubmissionError: sbatch: error: Batch job submission failed: Invalid qos specification
```

Developing and Debugging

Run Regression Tests (cont.)

Write some code…

```
36 function slurm_job_submit(job_desc, part_list, submit_uid)
37   -- PURPOSE: If no QoS specified, use default 'public'
38   if job_desc.qos == nil then
39     slurm.log_info("job submitted without qos")
40     job_desc.qos = "public"
41     slurm.log_user("Job updated to use qos: %s", job_desc.qos)
42   end
```

```
[root@dev-slurm01:slurm]# python3 test_slurm.py
.
----------------------------------------------------------------------
Ran 1 test in 0.101s

OK
```

SUCCESS! Tests Pass!

Developing and Debugging

Get Creative with Regression tests and make a lot!

```
@common_slurm_checks
@expect_tres({"cpu": 1, "mem": 2, "node": 1})
@expect_equal({"qos": "public"})
@expect_equal({"partition": ("htc", "general")})
@expect_equal({"features": "public"})
@expect_equal({"time_limit": 240})
def test_sbatch_multipart_htc_general_240(self):
    self.details = run('-p htc,general -t 240')

@raise_with_message(SlurmSubmissionError, "Job should be rejected/no jobid; any job with htc must be <= 240.")
def test_sbatch_multipart_htc_general_555(self):
    self.details = run('-p htc,general -t 555')

@raise_with_message(SlurmSubmissionError, "Job should be rejected/no jobid; lightwork and general are not compatible as a partition pair.")
def test_sbatch_multipart_lightwork_general(self):
    self.details = run('-p lightwork,general')
```

```
[root@phx-slurm01:slurm]# python3 test_slurm.py
ss............sss.....................................ssssss
----------------------------------------------------------------------
Ran 60 tests in 1.270s

OK (skipped=11)
```

Job Submit Plugin

To conclude:

- Unit Testing allows you to **declare your assumptions** about how the job submission process should work.
- Your job submit **plugin implements** those constraints.
- Run your JSP to **test those assumptions** and check for regressions.

Job Submit Plugin

Thank you!

Any questions?

Job Submit Plugin Lab

- Enable the Slurm API
- Enable the Job Submit Plugin with Lua
- [if you're adventurous] Create unit tests to check site business logic
- Write code to implement business logic

Job Submit Plugin Lab

Enable the Slurm API

The job submit plugin requires simply for the `slurm.conf` to be updated and `scontrol reconfig`'ed. However, unit-testing component requires the Slurm API to be operational.

As the Slurm API will also be used in a future lab, the minimum we need to do are enable these functionalities, even if we do not write job submit logic to change our scheduler behavior.

Create the containing directory in the Controller State Directory:

```
# install -d -o slurm -g slurm -m 0755 /var/spool/slurm.state/statesave
```

Create the key file then write 32 bytes (256 bits) of random data into it:

```
# install -o slurm -g slurm -m 0600 /dev/null /var/spool/slurm.state/statesave/jwt_hs256.key

# dd if=/dev/urandom of=/var/spool/slurm.state/statesave/jwt_hs256.key bs=32 count=1
status=none
```

Job Submit Plugin Lab

Enable the Slurm API

Now that the Java Web Token (JWT) is created, we enable it in the Slurm configuration. Add the following lines to `slurm.conf`:

```
AuthAltTypes=auth/jwt
AuthAltParameters=jwt_key=/var/spool/slurm.state/statesave/jwt_hs256.key,disable_token_cr
eation
```

This tells Slurm the keyfile in which to create the tokens needed for the Slurm API. To engage these changes, we must also:

```
scontrol reconfig
```

Job Submit Plugin Lab

Test `slurmrestd` is operational:

```
[root@lci-head-XX-1 slurm]# systemctl restart slurmrestd
[root@lci-head-XX-1 slurm]# systemctl status slurmrestd
● slurmrestd.service - Slurm REST daemon
     Loaded: loaded (/etc/systemd/system/slurmrestd.service; enabled; preset: disabled)
     Active: active (running) since Wed 2025-10-15 14:59:56 CDT; 13min ago
   Main PID: 59152 (slurmrestd)
      Tasks: 21 (limit: 48649)
     Memory: 4.4M
        CPU: 24ms
     CGroup: /system.slice/slurmrestd.service
             └─59152 /opt/slurm/current/sbin/slurmrestd 0.0.0.0:6820

…
Oct 15 15:11:16 lci-head-XX-1.novalocal slurmrestd[59152]: rest_auth/jwt:
slurm_rest_auth_p_authenticate: [[localhost]:42010(fd:9)] attempting token authentication
pass through
```

And finally, do we get a token when we request one with `scontrol token`?

```
[root@lci-head-01-1 slurm]# scontrol token
SLURM_JWT=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NjA1NjA5OTcsImlhdCI6MTc2MDU1OT
E5Nywic3VuIjoicm9vdCJ9.ZPhqrmyBPveDF0SU_gSO6JwG5091sO_o1GulsNxxH40
```

Job Submit Plugin Lab

Ensure `job_submit.lua` will be used on each submission:

```
[root@lci-head-XX-1 slurm]# grep JobSubmit /etc/slurm/slurm.conf
#JobSubmitPlugins=lua
```

^ Commented out, JSP is not engaged! Remove the comment:

```
JobSubmitPlugins=lua
```

and reconfig:

```
[root@lci-head-XX-1 slurm]# scontrol reconfig
```

Job Submit Plugin Lab

Start with the skeleton files:

```
$ sudo cp /home/rocky/slurm-quickstart/jsp/* /etc/slurm/
```

Test the basic skeleton works right away:

```
# cd /etc/slurm
# python3 test_jsp.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.024s

OK
```

Job Submit Plugin Lab

Please note, JSP is a very finicky, and considerably advanced topic on Slurm. Those without programming experience may find it difficult to follow and the syntax unforgiving.

To help alleviate the difficulty of this lab, I will mention Simplified Lab Instructions.

When Simplified Lab Instructions are listed on a slide, you can replace any actions described on that slide with the alternative instructions, even if it tells you to take no action.

It is the hope that the simplified lab instructions will help you see the process behind the iterative development of the Job Submit Plugin, without requiring the meticulous attention required to do test-driven development.

Job Submit Plugin Lab

Take a moment to review the `job_submit.lua` plugin as well as the unit test. Get familiar with the code to understand that:

1. Execution starts at "`if __name__ ==`"
2. Unit tests run all methods starting with "`test_`" in TestSlurm
3. Each test runs **sbatch** with the indicated "`run()`" params
4. Test are considered successful if they do not fail assertions, e.g., **`common_slurm_checks, expect_tres, expect_equal, raise_with_message`**
5. Of the two files, logic development should happen in **`job_submit.lua`**
6. `test_slurm.py` should contain *expectations of behavior*

Job Submit Plugin Lab

Understand the iterative process of Test-Driven Development:

1. **Write/revise a test to model the intended behavior.**
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. Write new code in `job_submit.lua`, rerun the test suite
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #1 Implement a default email domain for the user

Purpose: Ensure email updates reach user if user does not specify them directly.

Current behavior: user does not specify "`--mail-user=`" and users will not get start/end emails

Desired behavior: unspecified value shall default to:

```
--mail-user=<user>@example.com
```

Job Submit Plugin Lab

JSP Update #1 Implement a default email domain for the user

Write the assertion that an unspecified mail-to address should be replaced with their `username@example.edu`.

```
# Mail user asserted to be "<USER>@example.edu"
self.assertEqual(self.details['mail_user'], f"{getenv('USER')}@example.edu")
```

This is currently in-place on line ~107 of `test_jsp.py`. Simply uncomment it.

For this example, we want this to be universal across all job submissions, so I am putting it in a “common” place, the **`@common_slurm_checks`** testing functionality.

For the simplified lab, no action is necessary on this page.

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. **Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.**
3. Write new code in `job_submit.lua`, rerun the test suite
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #1 Implement a default email domain for the user

```
[root@lci-head-XX-1 slurm]# cd /etc/slurm
[root@lci-head-XX-1 slurm]# python3 test_jsp.py
.F
======================================================================
FAIL: test_sbatch_with_common_checks (__main__.TestSlurm)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/etc/slurm/test_jsp.py", line 108, in wrapper
    self.assertEqual(self.details['mail_user'], f"{getenv('USER')}@example.edu")
AssertionError: 'root' != 'root@example.edu'
- root
+ root@example.edu


----------------------------------------------------------------------
Ran 2 tests in 0.023s

FAILED (failures=1)
```

\- what we got

\+ what we expected

Yep, it does not work, as expected.

Simplified Lab Instructions, the test code has been saved in the file in advance:

```
# cd /etc/slurm
# python3 test_jsp_upd1.py
```

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. **Write new code in `job_submit.lua`, rerun the test suite**
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #1 Implement a default email domain for the user

```
[root@lci-head-01-1 slurm]# cat job_submit.lua
-- job_submit.lua
function slurm_job_submit(job_desc, part_list, submit_uid)

    --PURPOSE: detect unspecified mail user and substitute with username@example.com
    if job_desc.mail_user == nil then
        job_desc.mail_user = job_desc.user_name .. "@example.edu"
    end

    return slurm.SUCCESS
end

function slurm_job_modify(job_desc, job_rec, part_list, modify_uid)
    return slurm.SUCCESS
end
```

Add this new code in bold to `job_submit.lua`.

Simplified Lab Instructions, the LUA code has been saved in the file in advance:

```
# cp /etc/slurm/job_submit_upd1.lua /etc/slurm/job_submit.lua
```

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. Write new code in `job_submit.lua`, rerun the test suite
4. **When the test passes OK, save your work (git, preferably)!**
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #1 Implement a default email domain for the user

```
[root@lci-head-01-1 slurm]# python3 test_jsp.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.023s

OK
```

And we're done! Business logic given, business logic implemented.

Simplified Lab Instructions, re-run the test code again:

```
# python3 test_jsp_upd1.py
```

Take an extra moment to read these files, `test_jsp_upd1.py` and `job_submit_upd1.lua` to see why the inclusion of this code has accomplished what it intended.

Job Submit Plugin Lab

Note about the esoteric code we are reviewing and modifying:

```
# Mail user asserted to be "<USER>@example.edu"
self.assertEqual(self.details['mail_user'], f"{getenv('USER')}@example.edu")
```

What **is** `self.details`? How would I know what is in this?

`self.details` is a dictionary of values that the `slurmrestd` daemon returns to the calling python program, `test_slurm.py`. It contains all the characteristics of the submitted job as Slurm knows it, and allows us to check that values change per our business logic. **`self.details` is an implementation creately wholly for this exercise** and is not an official schedmd construct or even a well-known pattern.

Outside this lab, nobody would know what `self.details` is or how to use it. That is left up to the programmer at each site to decide how deeply they wish to get involved in testing their Job Submit Plugin functionality using this test-driven methodology. They can do so by understanding *this* more fully, or by writing their own.

Job Submit Plugin Lab

Note about the esoteric code we are reviewing and modifying:

We can try to make sense of what `self.details` includes if we just print it out:

```
# Basic test that checks a job is queued properly with basic conditions met
@common_slurm_checks
def test_sbatch_with_common_checks(self):
    self.details = run('-p general')
    print(self.details)
```

We can see the structure it returns is a basic python dictionary!

.{'account': 'root', 'accrue_time': {'set': True, 'infinite': False, 'number': 0}, 'admin_comment': '', 'allocating_node': 'lci-head-01-1', 'array_job_id': {'set': True, 'infinite': False, 'number': 0}, 'array_task_id': {'set': False, 'infinite': False, 'number': 0}, 'array_max_tasks': {'set': True, 'infinite': False, 'number': 0}, 'array_task_string': '', 'association_id': 2, 'batch_features': '', 'batch_flag': True, 'batch_host': '', 'flags': ['USING_DEFAULT_QOS', 'USING_DEFAULT_WCKEY'], 'burst_buffer': '', 'burst_buffer_state': '', 'cluster': 'lciadv', 'cluster_features': '', 'command': '', 'comment': '', 'container': '', 'container_id': '', 'contiguous': False, 'core_spec': 0, 'thread_spec': 32766, 'cores_per_socket': {'set': False, 'infinite': False, 'number': 0}, 'billable_tres': {'set': False, 'infinite': False, 'number': 0.0}, 'cpus_per_task': {'set': True, 'infinite': False, 'number': 1}, 'cpu_frequency_minimum': {'set': False, 'infinite': False, 'number': 0}, 'cpu_frequency_maximum': {'set': False, 'infinite': False, 'number': 0}, 'cpu_frequency_governor': {'set': False, 'infinite': False, 'number': 0}, 'cpus_per_tres': '', 'cron': '', 'deadline': {'set': True, 'infinite': False, 'number': 0}, 'delay_boot': {'set': True, 'infinite': False, 'number': 0}, 'dependency': '', 'derived_exit_code': {'status': ['SUCCESS'], 'return_code': {'set': True, 'infinite': False, 'number': 0}, 'signal': {'id': {'set': False, 'infinite': False, 'number': 0}, 'name': ''}}, 'eligible_time': {'set': True, 'infinite': False, 'number': 1760564584}, 'end_time': {'set': True, 'infinite': False, 'number': 1760578984}, 'excluded_nodes': '', 'exit_code': {'status': ['SUCCESS'], 'return_code': {'set': True, 'infinite': False, 'number': 0}, 'signal': {'id': {'set': False, 'infinite': False, 'number': 0}, 'name': ''}}, 'extra': '', 'failed_node': '', 'features': '', 'federation_origin': '', 'federation_siblings_active': '', 'federation_siblings_viable': '', 'gres_detail': [], 'group_id': 0, 'group_name': 'root', 'het_job_id': {'set': True, 'infinite': False, 'number': 0}, 'het_job_id_set': '', 'het_job_offset': {'set': True, 'infinite': False, 'number': 0}, 'job_id': 34, 'job_resources': {}, 'job_size_str': [], 'job_state': ['PENDING'], 'last_sched_evaluation': {'set': True, 'infinite': False, 'number': 1760564583}, 'licenses': '', 'mail_type': [], 'mail_user': 'root@example.edu', 'max_cpus': {'set': True, 'infinite': False, 'number': 0}, 'max_nodes': {'set': True, 'infinite': False, 'number': 0}, 'mcs_label': '', 'memory_per_tres': '', 'name': 'wrap', 'network': '', 'nodes': '', 'nice': 0, 'tasks_per_core': {'set': False, 'infinite': True, 'number': 0}, 'tasks_per_tres': {'set': True, 'infinite': False, 'number': 0}, 'tasks_per_node': {'set': True, 'infinite': False, 'number': 0}, 'tasks_per_socket': {'set': False, 'infinite': True, 'number': 0}, 'tasks_per_board': {'set': True, 'infinite': False, 'number': 0}, 'cpus': {'set': True, 'infinite': False, 'number': 1}, 'node_count': {'set': True, 'infinite': False, 'number': 1}, 'tasks': {'set': True, 'infinite': False, 'number': 1}, 'partition': 'general', 'prefer': '', 'memory_per_cpu': {'set': True, 'infinite': False, 'number': 2048}, 'memory_per_node': {'set': False, 'infinite': False, 'number': 0}, 'minimum_cpus_per_node': {'set': True, 'infinite': False, 'number': 1}, 'minimum_tmp_disk_per_node': {'set': True, 'infinite': False, 'number': 0}, 'power': {'flags': ['preemptable_time': {'set': True, 'infinite': False, 'number': 0}, 'pre_sus_time': { 'hold': False, 'priority': {'set': True, 'infinite': False, 'number': 50000000}, 'profile': ['NOT_SET'], 'qos': 'normal', 'reboot': False, 'required_nodes': '', 'minimum_switches': 0, 'requeue': True, 'resize_time': {'set': True, 'infinite': False, 'number': 0}, 'restart_cnt': 0, 'resv_name': '', 'shared': [], 'exclusive': [], 'oversubscribe': True, 'show_flags': [], 'sockets_per_board': 0, 'sockets_per_node': {'set': False, 'infinite': False, 'number': 0}, 'start_time': {'set': True, 'infinite': False, 'number': 1760564584}, 'state_description': '', 'state_reason': 'BeginTime', 'standard_error': '/etc/slurm/slurm-34.out'

Job Submit Plugin Lab

JSP Update #2 Full mem requests get whole node

Purpose: Users that request all the memory should get charged for all resources on the node they make unavailable

Current behavior: Users get 100% via `--mem=0` and `-c 1` for only 1 node, meaning idle, unusable cores are merely ignored from an accounting standpoint

Desired behavior: automatically append `--exclusive` functionality on job submission if 100% of memory is requested.

Job Submit Plugin Lab

1. **Write/revise a test to model the intended behavior.**
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. Write new code in `job_submit.lua`, rerun the test suite
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #2 Full mem requests get whole node

Write the assertion into `test_jsp.py` that any request must take the whole node.

```
# when requesting 100% of memory, indicate the whole node as --exclusive (shared: 2)
@expect_equal({"shared": ['user']})
def test_sbatch_exclusivity(self):
    self.details = run('-p general --mem=0')

@expect_equal({"shared": []})
def test_sbatch_not_exclusivity(self):
    self.details = run('-p general --mem=1G')
```

These are two new tests which checks that all jobs (in our example case) **must have the whole node** if `--mem=0` is used, regular behavior otherwise.

For the simplified lab, no action is necessary on this page.

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. **Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.**
3. Write new code in `job_submit.lua`, rerun the test suite
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #2 Full mem requests get whole node

```
[root@lci-head-01-1 slurm]# python3 test_jsp.py
.F..
======================================================================
FAIL: test_sbatch_exclusivity (__main__.TestSlurm)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/etc/slurm/test_jsp.py", line 171, in wrapper
    self.assertEqual(self.details[key], expected_value)
AssertionError: Lists differ: [] != ['user']

Second list contains 1 additional elements.
First extra element 0:
'user'

- []
+ ['user']

----------------------------------------------------------------------
Ran 4 tests in 0.048s

FAILED (failures=1)
```

Note 3 successes: the _not_exclusivity test is behaving *as expected*.

Simplified Lab Instructions, the test code has been saved in the file in advance:

```
# cd /etc/slurm
# python3 test_jsp_upd2.py
```

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. **Write new code in `job_submit.lua`, rerun the test suite**
4. When the test passes OK, save your work (git, preferably)!
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #2 Full mem requests get whole node

```
[root@lci-head-01-1 slurm]# cat job_submit.lua
-- job_submit.lua
function slurm_job_submit(job_desc, part_list, submit_uid)
    <…previous code…>
    -- PURPOSE: requests for all memory engages --exclusive
    if (job_desc.min_mem_per_node == 0) then
      job_desc.shared = 2
    end

    return slurm.SUCCESS
end

function slurm_job_modify(job_desc, job_rec, part_list, modify_uid)
    return slurm.SUCCESS
end
```

Add the code in bold to `job_submit.lua`.

Simplified Lab Instructions, the LUA code has been saved in the file in advance:

```
# cp /etc/slurm/job_submit_upd2.lua /etc/slurm/job_submit.lua
```

Job Submit Plugin Lab

1. Write/revise a test to model the intended behavior.
2. Ensure the current implementation does not already do this (accidentally!) by running the test with a FAILING result.
3. Write new code in `job_submit.lua`, rerun the test suite
4. **When the test passes OK, save your work (git, preferably)!**
5. If desired, refactor your code–do not change your tests.
6. When the test passes OK, save your work!
7. Repeat for all intended behavior.

Job Submit Plugin Lab

JSP Update #2 Full mem requests get whole node

```
[root@lci-head-01-1 slurm]# python3 test_jsp.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.041s

OK
```

SUCCESS!

Simplified Lab Instructions, re-run the test code again:

```
# python3 test_jsp_upd2.py
```

Take an extra moment to read these files, `test_jsp_upd2.py` and `job_submit_upd2.lua` to see why the inclusion of this code has accomplished what it intended.

Job Submit Plugin Lab

Remember:

- Unit Testing allows you to **declare your assumptions** about how the job submission process should work.
- Your job submit **plugin implements** those constraints.
- Run your JSP to **test those assumptions** and check for regressions.